Compiler-Guided Repair for Natural-Language-to-Lean Formalization

Sadia Peerzada

sadiapeerzada2006@gmail.com

Abstract. We execute a paired local-model study of one-shot proof generation (A), blind repair (B), and compiler-guided repair (C) for natural-language statements paired with fixed Lean theorem types. B and C reuse exactly the same failed initial candidate and permit at most three repairs each. The frozen dataset contains 60 test statements and 15 development statements across five categories, screened under an explicitly AI-assisted semantic policy. Using one hash-pinned 4-bit Qwen2.5-7B instruction model, the observed verified success counts are A: 0/60 (0.0%), B: 1/60 (1.7%), C: 1/60 (1.7%). The C–B difference is +0.0 percentage points; the 95% family-cluster bootstrap interval is [+0.0, +0.0] points. The recorded experiment does not establish a robust advantage for compiler feedback; a non-significant or non-informative test is not evidence of equivalence. We preserve raw model responses, compiler diagnostics, token streams, stopping events and checkpoints. The evidence concerns fixed-type proof synthesis, not unrestricted generation of faithful formal statements. Single-seed inference, related theorem families, author-assisted AI adjudication, and library/pretraining exposure limit generalization.

Keywords: Lean 4 · compiler feedback · proof repair · local inference · semantic review

1 Introduction and Research Questions

An informal mathematical statement can be paired with a formal theorem type yet still lack a proof accepted by a theorem prover. A generated proof may fail because of syntax, unavailable names, type mismatches, an incomplete tactic argument, or a mathematical mistake. A compiler supplies concrete evidence about these failures, but additional generation alone may also repair them. A useful comparison must distinguish diagnostic feedback from the benefit of more calls.

Our primary question is whether compiler-guided repair improves verified success over equally budgeted blind repair, conditioned on the same initial candidate. Secondary questions compare each repair condition with one-shot generation and describe outcomes by initial error category. We do not assume that feedback helps, or interpret a failure to reject as equivalence. The experimental unit is a theorem trajectory, not an individual repair attempt.

The evaluated stage receives both English and a fixed AI-reviewed Lean type. It therefore tests proof synthesis and repair within an NL-to-Lean workflow. It does not measure unconstrained English-to-theorem translation, independent semantic-review accuracy, or state-of-the-art mathematical problem solving.

2 Related Work

Lean 4 provides a dependently typed language and kernel-checked proof infrastructure [4]; Mathlib supplies the mathematical library used here [6]. Benchmarks such as miniF2F [8] support formal theorem-proving evaluation, while ProofNet studies paired informal and formal mathematics [1]. LeanDojo studies tool-supported theorem proving and retrieval [7]. These settings motivate careful separation of problem statements, proof generation, library access and verification. Our experiment is a bounded paired comparison using a general local instruction model. It includes no retrieval service, proof search tree, fine-tuning, or paid hosted inference, and should not be compared directly with systems using different tasks, data or compute budgets.

3 System Architecture and Verification

The pipeline loads immutable English/type pairs, renders an initial prompt, generates one candidate, extracts the requested Lean block, and verifies it against the fixed reference type. A and both repair branches share the exact initial bytes and SHA-256. Each repair uses a fresh model conversation and independent context-derived random stream. B receives the theorem, its current candidate and fixed blind-feedback text. C receives the same template structure with structured diagnostics from its immediately preceding failed compilation. Neither branch receives the other branch's candidates or conversation.

The strict verifier rejects placeholder proofs and unauthorized declarations, checks the exact elaborated theorem type, and inspects target axiom dependencies. Only the standard `propext`, `Classical.choice` and `Quot.sound` dependencies are accepted. A reference placeholder is used only to compare types in a separate namespace; it is never counted as a target proof, and a target depending on `sorryAx` is rejected. Compiler JSON, source text, exit status, failure kind and elapsed time are stored. Compilation proves neither the English interpretation nor the quality of explanatory prose.

Source-token and declaration guards run before compilation; axiom-policy rejection can also occur afterward. Such events are not relabeled as compiler errors. C stops if genuine candidate diagnostics are unavailable; it is never supplied an invented compiler message. Timeouts, context-limit failures and interrupted calls remain explicit missing/incomplete outcomes.

Table 1. Frozen AI-policy dataset composition. Development has five curated family clusters and test has 54; no family crosses the split.

Category	Dev.	Test	Total
Arithmetic identities	2	11	13
Divisibility	3	17	20
Induction	6	8	14
Inequalities	1	15	16
Modular arithmetic	3	9	12
Total	15	60	75
Difficulty 1	2	2	4
Difficulty 2	10	57	67
Difficulty 3	2	1	3
Difficulty 4	1	0	1
Textbook	8	8	16
Formal library	7	52	59

4 Dataset, Provenance and Semantic Audit

Starting from 96 prepared candidates, we exclude one exact duplicate and 20 nonduplicate records lacking adequate item-level provenance. The resulting 75 records comprise 16 textbook-derived statements [2] and 59 adaptations of pinned formal-library statements. Citations, original source bytes, license notices and item-level attribution are retained. Textbook adaptations retain CC BY-SA 4.0 attribution; formal-library material retains Apache-2.0 notices. These are evidence-based AI-assisted source assessments, not invented publisher permissions or independent legal clearance.

Two original records were corrected: a positive-modulus premise was restored, and ambiguous summation endpoints were made explicit. Both proofs were reverified and both pairs were reviewed again with a separate Phi-4 model. They remain excluded for source-provenance reasons; corrections were not used to inflate eligible counts. Original records and before/after histories remain available.

The approved replacement policy uses explicit AI adjudication of quantifiers, domains, assumptions, strength, endpoints and equivalent encodings. Phi-3.5 and Phi-4 audit responses are retained, including schema failures and unsupported objections. The authoring coding assistant adjudicated the exact selected pairs using candidate-specific mathematical comparisons and reference evidence. It is not author-independent human review. An uncertain or unfaithful final adjudication is inadmissible; unreliable raw AI output is not a faithful vote.

The independent reviewer models were demonstrably fallible: for example, one claimed that $1(1 + 1)$ was odd. Exact arithmetic refutes that alleged counterexample. Generic placeholders, hallucinated hypotheses and proof-method confusion were separately flagged. All 96 original reference types have strict Lean evidence; handwritten bounded probes cover 26,467 accepted finite assignments, with 88 exact predicates and eight approximate numerical probes. These checks supplement, rather than establish, English equivalence.

Whole binomial, linear-solvability and modular-power families were grouped before primary-model test access. No curated family crosses splits. The 15 development statements occupy five conservative family clusters; the 60 test statements occupy 54. These are not claimed to be 75 independent topic families. All texts were accessed for authorized curation, but no held-out primary-model proof outcome was used to select prompts or settings. Ordered IDs and exact dataset bytes were frozen with cryptographic hashes.

5 Frozen Experimental Design

The primary model is `mlx-community/Qwen2.5-7B-Instruct-4bit`, run offline with MLX [5, 3]. Its immutable converted snapshot is `c26a38f6a37d0a51b4e9a1eb3026530fa35d9fed`. Inventory hashes bind all weights, tokenizer files and the native chat template. Quantization is 4-bit with group size 64. MLX 0.32.4, MLX-LM 0.32.2 and Transformers 5.21.0 are recorded. Decoding uses temperature 0.2, top-p 1, at most 2,000 output tokens and an 8,192-token total context cap. One primary seed, 2027, is used; three-seed robustness is not claimed. The local host uses an Apple M4 processor with 16 GiB physical memory.

The per-call seed is the first unsigned 32-bit word of SHA-256 of the base seed, phase/problem/condition/step context and exact prompt hash. Consequently, B's stopping behavior does not shift C's random stream. All conditions use the same model and environment. A has zero repairs; B and C each permit up to three, with at most seven logical generations per problem including the shared initial call. Success stops a branch. Model weights are released during the memory-heavy Lean check and reloaded from the same verified bytes on the next call. This preserves independent KV state and incurs measurable loading overhead.

Exploratory development pilots exposed Lean 3-style syntax and high resident memory cost. Before any held-out generation, the final protocol added the same explicit Lean 4 syntax instruction to all three conditions and fixed the memory schedule. This was syntax/implementation development, not test-outcome tuning; the format diagnostic still failed mathematical verification. Prototypes are preserved and excluded from study estimates. The final three-case pilot passed the provenance/control audit, and an actual resume invocation generated no new calls. No model, prompt, dataset or budget changed after held-out execution began.

Conditions execute B before C rather than in randomized order. Each call has a durable intent, raw response and token stream; each verification and branch transition has a checkpoint. Resume reuses completed outputs. An incomplete generation intent consumes its logical call and is not silently regenerated. The environment is Lean 4.32.0 with clean Mathlib commit `81a5d257c8e410db227a6665ed08f64fea08e997`. There is no provider fallback or paid inference path.

Table 2. Observed strict proof success; full eligible denominator, counting missing outcomes as failures. Wilson intervals assume problem independence. Decode-token totals include the shared initial for each counterfactual condition; physical totals count it once.

Condition	Success Rate	95% CI	Repairs	Output tokens
A	0/60 0.0%	[0.0, 6.0]	0	27,862
B	1/60 1.7%	[0.3, 8.9]	178	83,485
C	1/60 1.7%	[0.3, 8.9]	174	74,778

6 Results and Statistical Analysis

Across all 60 frozen test statements, observed verified success is A: 0/60 (0.0%), B: 1/60 (1.7%), C: 1/60 (1.7%). There are 2 incomplete trajectories. Table 2 uses the full eligible denominator, treating unavailable outcomes as failures for the conservative sensitivity estimate; incomplete outcomes are also reported separately in the artifacts. Figure 1 shows cumulative observed success against the repair budget. Repair calls are adaptive attempts, not independent samples or additional benchmark problems.

Unavailable condition outcomes, retained in the eligible denominator.

Condition	Recorded cause	Count
C	Context budget exceeded	1
C	Interrupted generation	1

The primary C–B comparison has 58 complete matched pairs. Its full-denominator difference is +0.0 percentage points, with family-cluster 95% interval [+0.0, +0.0]. The exact two-sided McNemar p is non-informative; the Holm-adjusted value over three planned comparisons is non-informative. The recorded experiment does not establish a robust advantage for compiler feedback; a non-significant or non-informative test is not evidence of equivalence. Problem-level Wilson intervals and exact McNemar tests assume independent problem units; related within-family targets limit that assumption. We therefore report the pre-specified family-cluster sensitivity interval and do not make an unqualified significance claim. Zero discordance is non-informative; a degenerate bootstrap interval would not prove equivalence.

Paired differences in percentage points (pp), family-cluster bootstrap intervals, exact McNemar tests on complete pairs, and Holm adjustment over three comparisons. Cluster dependence limits the problem-level tests.

Comparison	Diff. pp	Cluster 95% CI	p	p_{Holm}
C_vs_B	+0.0	[+0.0, +0.0]	non-informative	non-informative
C_vs_A	+1.7	[+0.0, +5.5]	1	1
B_vs_A	+1.7	[+0.0, +5.5]	1	1

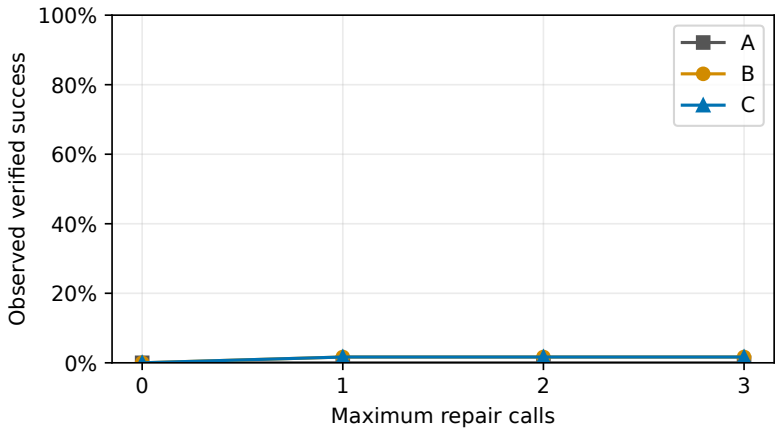


Fig. 1. Observed verified success by maximum repair budget. The budget-zero point is identical across all conditions because the initial candidate is shared. Missing outcomes count as failures in this full-denominator display.

Intervals use 10,000 bootstrap resamples and seed 20271021, at problem and family-cluster levels. Category summaries and error-conditioned outcomes are descriptive, with actual denominators; no post-hoc subgroup significance claim is made. The physical run used 412 generation attempts and 410 completed generations, with 362,035 known prompt tokens and 130,401 completion tokens from completed generations. Partial streams additionally preserve 0 observed generated tokens; those are not completed responses and are excluded from the completed-call totals. Physical totals count the initial call once; each counterfactual condition's resource total includes that shared initial cost. Raw wall, generation and compiler timings are retained. Generation timings exclude model-loading overhead; trajectory wall times include it. Sequential condition order, cache state and host activity limit causal latency comparisons. A read-only process observation during execution also identified a separate local MLX workload on the same host; its duration and attributable overhead were not measured, and the unrelated workload was not modified. A worker interruption required checkpoint recovery. The incomplete generation was not replayed. Trajectory elapsed time for that case includes the inactive interval before recovery, so it cannot be interpreted as continuous compute time. The resource matrix reports known input/output tokens, model-call wall seconds (including reload overhead), and Lean seconds. An interrupted call without a recorded endpoint has no invented duration; its trace and missingness remain explicit in the supplementary resource data.

Condition resource totals (including shared initial).

Cond.	Input tokens	Output tokens	Call s	Lean s
A	12,929	27,862	1908.6	2361.9
B	122,029	83,485	8120.5	8923.9
C	252,935	74,778	9875.1	8906.3

Completed-call resource distributions. Tokens include input and output; call wall time includes loading. Each condition includes the shared initial. Interrupted calls are excluded, not assigned zero.

Cond.	Measure	Calls	Mean	Median
A	Total tokens	60	679.9	425.5
A	Generation s	60	27.2	11.8
A	Call wall s	60	31.8	16.3
B	Total tokens	238	863.5	540.5
B	Generation s	238	26.1	12.3
B	Call wall s	238	34.1	16.9
C	Total tokens	232	1412.6	1149.0
C	Generation s	232	28.7	16.6
C	Call wall s	232	42.5	21.0

Repair generation attempts per eligible problem (60-case denominator, including zero calls and consumed interrupted intents).

Condition	Total	Mean	Median
A	0	0.00	0.0
B	178	2.97	3.0
C	174	2.90	3.0

7 Compiler Errors and Qualitative Analysis

Table 3 groups failed initial candidates into a single primary category using a fixed message taxonomy; Figure 2 shows verified repair outcomes. Parser, unknown-name, type/elaboration and tactic/goal failures are distinguished. Safety guards and infrastructure failures are separate from compiler errors. The artifact also records all attempted-candidate error events; those repeated events are not independent problem observations. As a post-hoc process diagnostic, 90 of 178 completed B repairs and 70 of 172 completed C repairs were byte-identical to the preceding candidate. These counts describe repetition, not additional tests or independent samples.

The following 5 examples are authentic recorded outputs, selected post hoc to cover available outcome types, initial error classes and mathematical categories, then dataset order. Selection does not change the statistical sample. A blind-only success displays its last B output; otherwise the last available C output is displayed, followed by B or the shared initial when C made no attempt. The branch and verifier outcome are explicit. Code/diagnostic snippets are truncated and ASCII-transliterated for typesetting only; complete original Unicode bytes, prompts and diagnostics remain in the run artifacts.

Table 3. Repair outcomes grouped by the primary initial failure category (one category per failed initial proof). Guard/infrastructure failures are separated from genuine compiler errors. Small groups are descriptive.

Initial failure	Cases	B solved	C solved
Other compiler error	1	0	0
Parser / syntax	26	0	0
Tactic / goals	9	0	0
Type / elaboration	4	0	0
Unknown name	20	1	1

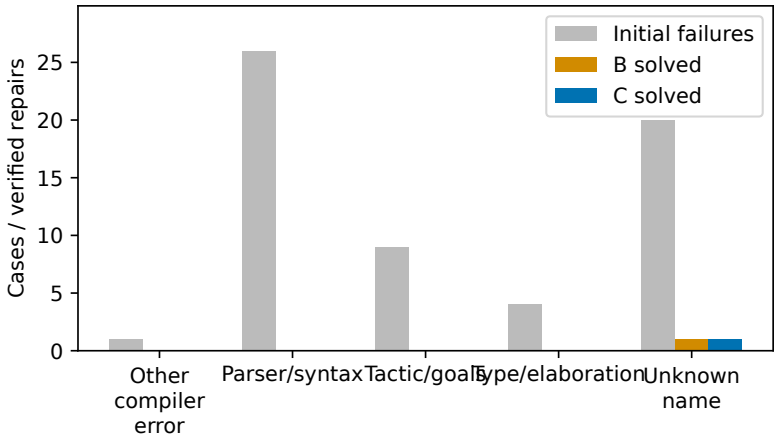


Fig. 2. Verified B/C repair outcomes by primary initial failure category. The denominator for each group is given in Table 3; small groups are illustrative rather than inferential.

lib_a_exp_add (both repair). For all real x and y, exp(x+y)=exp(x)*exp(y). A/B/C verified outcomes were False/True/True. Displayed candidate: C; verifier success: True. Initial diagnostic: line 13: Unknown constant ‘Real.ofReal‘; line 13: unsolved goals x y : Real |- Real.exp x * Real.exp y = sorry; line 15: Unknown constant ‘Real.ofReal‘; line 16: Unknown constant ‘Real.ofReal_exp‘.

```
open scoped BigOperators
theorem study_a_exp_add (x y : Real) : Real.exp (x + y) = Re
:= by
  have h : Real.exp (x + y) = Real.exp x * Real.exp y := by
    rw [Real.exp_add]
  exact h
```

div_007 (both failed). For all integers a and b, if a and b are even, then a + b is even. A/B/C verified outcomes were False/False/False. Displayed candidate: C; verifier success: False. Initial diagnostic: line 13: Unknown identifier ‘even_add‘.

```
theorem even_add_of_even (a b : Int) (ha : Even a) (hb : Eve
by
  obtain <a', ha'> := ha
  obtain <b', hb'> := hb
  exact even_add ha' hb'
```

lib_a_trig_unit (incomplete). For every real x, sin(x)^2+cos(x)^2 equals one. A/B/C verified outcomes were False/False/None. Displayed candidate: B; verifier success: False. Initial diagnostic: line 10: unexpected token ’‘’; expected command; line 38: unexpected token ’end’; expected ’)’, ’,’ or ’:’; line 14: unsolved goals x : Real |- 1 = Real.sin x * Real.sin x + Real.cos x * Real.cos x; line 18: un.

```
open scoped BigOperators
theorem study_a_trig_unit (x : Real) : (Real.sin x) ^ 2 + (R
:= by
  have h1 : (Real.sin x) ^ 2 + (Real.cos x) ^ 2 = (Real.sin
    simp
  have h2 : (Real.sin x) * (Real.sin x) + (Real.cos x) * (Re
    simp
```

div_008 (both failed). Prove that for all integers n, it is the case that n is even if and only if 3n is even. That is, prove both implications: if n is even, then 3n is even, and if 3n is even, then n is even. A/B/C verified outcomes were False/False/False. Displayed candidate: C; verifier success: False. Initial diagnostic: line 14: unknown tactic.

```
theorem even_iff_even_three_mul (n : Int) : Even n <-> Even
by
  split
  ? intro h
    obtain <k, hk> := h
    exact even_of_add_one_even (by rw [mul_one, hk, mul_one]
```

div_011 (both failed). For integers a, b, and c, if a divides both b and c, then a divides both b + c and b - c. A/B/C verified outcomes were False/False/False. Displayed candidate: C; verifier success: False. Initial diagnostic: line 13: Type mismatch k + l has type Int of sort ‘Type‘ but is expected to have type a | b + c of sort ‘Prop‘; line 13: Tactic ‘rewrite‘ failed: Did not find an occurrence of the pattern ?a * (?b + ?.

```
theorem dvd_add_sub_int (a b c : Int) (hab : a  |  b) (hac :
```

Table 4. Semantic and correctness evidence. AI adjudication is an eligibility decision, not independently measured semantic accuracy. No independent human semantic review is claimed.

Evidence	Count	Interpretation
AI-adjudicated references	75/75	Policy-screened, not human gold
Independent human review	0/75	Student-reported substitution
Raw Phi-4 faithful labels	72/96	Before adjudication; not accuracy
Raw Phi-4 invalid format	13/96	Not accepted as faithful votes
Corrected original pairs	2	Reviewed again; excluded for provenance

```
:= by
  obtain <k, hk?> := hab
  obtain <l, hl?> := hac
  exact <k + l, k - l, by
    rw [mul_add, mul_sub]
    exact <k + l, by rw [mul_add, hk?, hl?]>
```

8 Semantic Faithfulness and Threats to Validity

Successful proofs must have the exact reviewed reference type; changing a quantifier, adding a hypothesis or weakening the conclusion cannot substitute for solving that target. Nevertheless, faithfulness to English remains conditional on the AI-adjudicated reference. We report review coverage and provenance, not independently measured semantic accuracy. No AI output is represented as an independent human judgment.

The authoring/adjudicating assistant can share systematic biases with the dataset construction process. Separate Phi reviewers were unreliable, and bounded mathematical probes are not exhaustive proofs of informal meaning. Source/library exposure and direct named-lemma reuse may affect performance; the dataset is not a contamination-free benchmark. Curated family boundaries are mathematical judgments, not guarantees of statistical independence. The development set has only five family clusters, and results use one model, one quantization, one primary seed and one local hardware setting. The model's knowledge can predate the pinned Lean/Mathlib interfaces. Fixed call and context caps can stop repair without exhausting mathematical possibilities. These limits preclude broad claims about compiler feedback, larger models or unrestricted NL-to-formal translation.

9 Reproducibility and Authorization Provenance

All results derive from the frozen protocol and ordered dataset, with durable raw generations, compiler JSON, exact sources, token counts, timings, failures and resume checkpoints. Protocol SHA-256 is `a032110d7afbc367c2daa820`

bd17511c6ff5dba2d1164e0130ea5d1c4b693317; dataset-freeze SHA-256 is de0850197b821ca4c519737c4cffb16d78e2a329d15be6c7f3c6a7f5487db67f; model inventory SHA-256 is 24d68abbe5e1434e5263ad123cca517c2c1799c7465e104e83f6cb02baeac4f9. The pre-held-out source commit and launch time, complete test outputs, source/evidence hashes and PDF build manifest are included. Original Groq and exploratory local protocols remain available for provenance.

The student reported supervisor approval of AI-assisted semantic review and free local MLX substitution on 28 October 2026. The original decision date, document and signature were not supplied; approval is not claimed independently verified. Exact implementation choices were frozen under the user's execution authorization. No publisher permission or human review was fabricated. AI tools assisted construction, adjudication and inference; they are not credited as authors. The supplied author name and correspondence are used without inventing an institutional affiliation.

10 Conclusion

This study records a controlled local comparison of one-shot, blind and compiler-guided proof repair for 60 fixed theorem targets with shared initial candidates. The recorded experiment does not establish a robust advantage for compiler feedback; a non-significant or non-informative test is not evidence of equivalence. The contribution is the reproducible paired evidence and its failure analysis under a clearly delimited low-resource configuration, not a claim of general superiority, independent human semantic validation, or complete unconstrained natural-language formalization.

References

1. Azerbayev, Z., Piotrowski, B., Schoelkopf, H., Ayers, E.W., Radev, D., Avigad, J.: ProofNet: Autoformalizing and formally proving undergraduate-level mathematics (2023), https://arxiv.org/abs/2302.12433
2. Levin, O.: Discrete mathematics: An open introduction (2021), https://discrete.openmathbooks.org/dmoi3.html, third edition, fifth printing. Source states CC BY-SA 4.0; source and license evidence archived
3. Mlx lm: Large language model inference and fine-tuning for apple silicon. https://github.com/ml-explore/mlx-lm, installed version 0.32.2; accessed 27 October 2026
4. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Automated Deduction – CADE 28. Lecture Notes in Computer Science, vol. 12699, pp. 625–635. Springer (2021). https://doi.org/10.1007/978-3-030-79876-5_37
5. Mlx conversion of qwen2.5-7b-instruct, four-bit. https://huggingface.co/mlx-community/Qwen2.5-7B-Instruct-4bit, snapshot c26a38f6a37d0a51b4e9a1eb3026530fa35d9fed; model card and Apache-2.0 license evidence archived 27 October 2026

6. The mathlib Community: The Lean mathematical library. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs. pp. 367–381. ACM (2020). https://doi.org/10.1145/3372885.3373824
7. Yang, K., Swope, A., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R.J., Anandkumar, A.: LeanDojo: Theorem proving with retrieval-augmented language models. In: Advances in Neural Information Processing Systems. vol. 36 (2023), https://proceedings.neurips.cc/paper_files/paper/2023/hash/4441469427094f8873d0fecb0c4e1cee-Abstract-Datasets_and_Benchmarks.html
8. Zheng, K., Han, J.M., Polu, S.: minif2f: A cross-system benchmark for formal olympiad-level mathematics. In: International Conference on Learning Representations (2022), https://arxiv.org/abs/2109.00110